EECS 31L: INTRODUCTION TO DIGITAL DESIGN LAB
LECTURE 3

Salma Elmalaki
salma.elmalaki@uci.edu

The Henry Samueli School of Engineering
Electrical Engineering and Computer Science
University of California, Irvine

LECTURE 3: LOGISTICS

- **All questions should be posted on canvas**
- **Please check the submission guidelines on canvas and in the first lecture**
- Lab 2 will be assigned right after Lab 1 deadline

Last time

- Verilog module declaration
- Numerical representation
- Verilog data types (reg, wire)

LECTURE 3: OVERVIEW

- Verilog datatype (revision from lecture 2)
- Verilog Operators
- Typical mistakes in Verilog
- Testbench
- Data flow description
- Concurrent signal assignment + examples
- Assignment with delay + examples
- Concurrent code Verilog
- There will be a 10-minute Quiz assigned on Canvas covering contents up till this lecture. Due Friday at 11:59pm (No Late submission)

SOME VERILOG DATA TYPES

- **Nets**
  - –Defined by predefined word *wire,* with 4 known values
  - –Can be driven in concurrent statements
  - –They change continuously by circuit
  - –Syntax: Example:

```
wire net_name;   //just name
wire S1= 1'b0;   //name and initial value
```

- **Registers**
  - –Defined by predefined word *reg,* with 4 known values
  - –Can be driven in sequential statements
  - –In contrast to nets stores values until they are updated
  - –Syntax: Example:

```
reg reg_name; //just name
reg S1= 1'b0; //name and initial values
```

4-state types

Value	Definition
0	Logic 0 (false)
1	Logic 1 (true)
X	Unknown
Z	High impedance

MORE VERILOG DATA TYPES

- **Vector**

Multiple bits.
A reg or a net can be declared as a vector.
Vectors are declared by brackets [ ]
Examples:

```
reg [7:0] MB1; //8-bit reg vector with MSB=7 LSB=0
wire [0:7] MB2; //8-bit wire vector with MSB=0 LSB=7
reg [3:0] bitslice;
 // with initialization
wire [3:0] multiBitWord1 = 4'b1010;
reg [7:0] total = 8'd12;  //8 bits and decimal value 12
                          // 8'b0000_1100
```

Referencing vectors

```
a = multiBitWord1[3];    // a = 1
bitslice1 = total[3:0];  // bitslice1 = 4'b1100
bitslice2 = total[7:4];  // bitslice2 = 4'b0000
```

MORE VERILOG DATA TYPES

•Vector

Multiple bits.
A reg or a net can be declared as a vector.
Vectors are declared by brackets [ ]
Examples:

```
reg [7:0] MB1; //8-bit reg vector with MSB=7 LSB=0
wire [0:7] MB2; //8-bit wire vector with MSB=0 LSB=7
reg [3:0] bitslice;
```

•Array

Array as per definition in a collection of elements of same type.

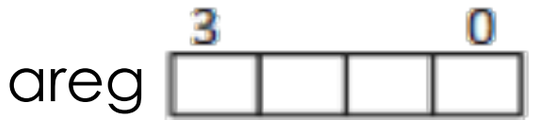
```
reg signed [3:0] anArray [0:4];
//anArray has 5 elements and each element is 4 signed
bits
```

Size of each element

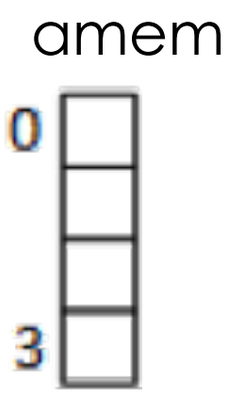
Size of the array

VERILOG DATA TYPES

```
//An 4-bit vector
wire [3:0] areg;
```

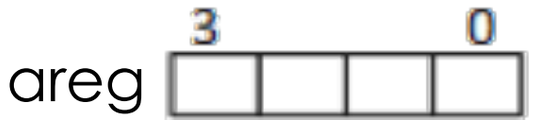



```
//A memory of 4 one-bit elements
reg amem [0:3];
```

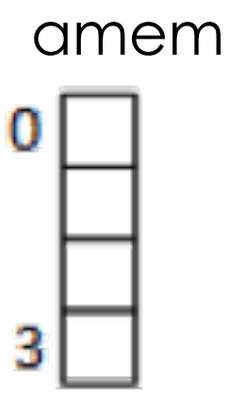


VERILOG DATA TYPES

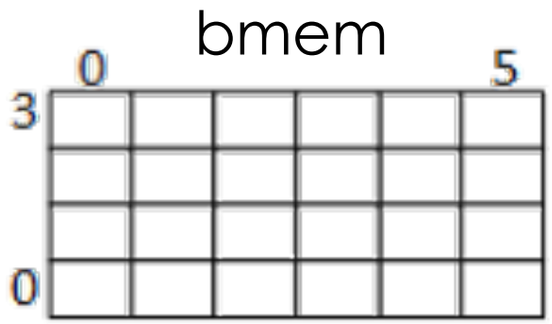
```
//An 4-bit vector
wire [3:0] areg;
```



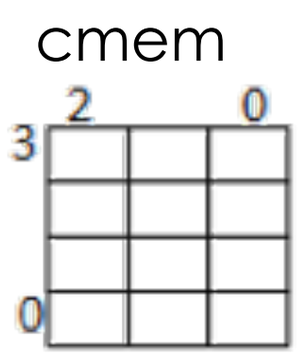

```
//A memory of 4 one-bit elements
reg amem [0:3];
```




```
//A memory of four 6-bit
reg [0:5] bmem [3:0];
```

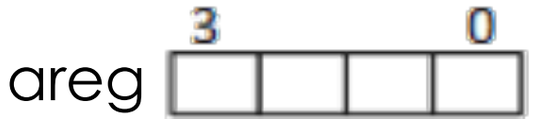



```
//A two dimensional memory of one-bit elements
reg cmem [3:0][2:0];
```

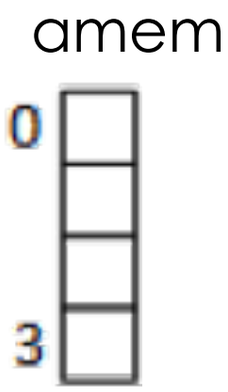


VERILOG DATA TYPES

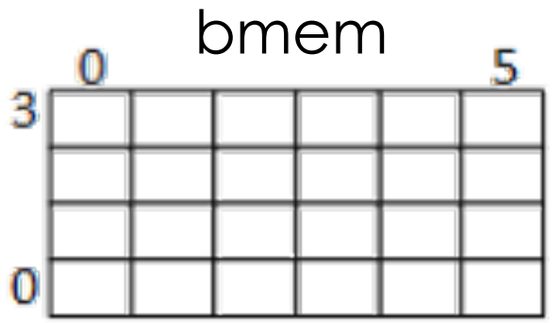
```
//An 4-bit vector
wire [3:0] areg;
```



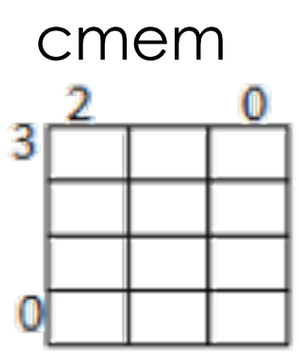

```
//A memory of 4 one-bit elements
reg amem [0:3];
```




```
//A memory of four 6-bit
reg [0:5] bmem [3:0];
```



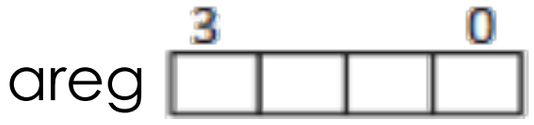

```
//A two dimensional memory of one-bit elements
reg cmem [3:0][2:0];
```



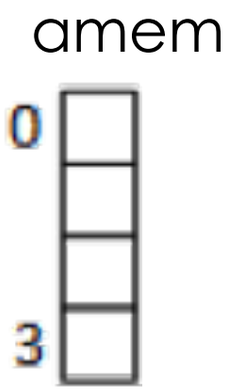
VERILOG DATA TYPES

- **Parameter**

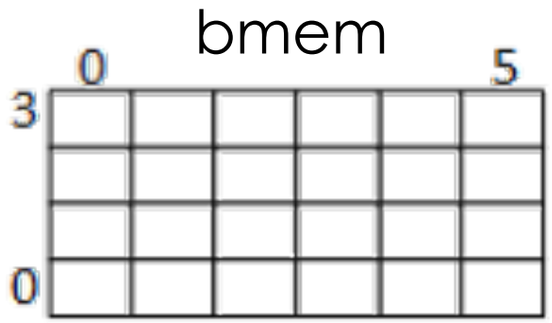
```
//An 4-bit vector
wire [3:0] areg;
```



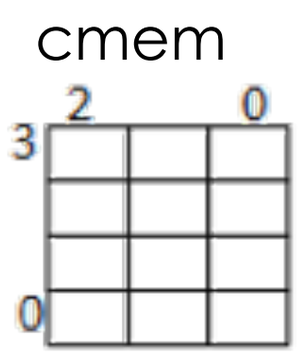

```
//A memory of 4 one-bit elements
reg amem [0:3];
```




```
//A memory of four 6-bit
reg [0:5] bmem [3:0];
```




```
//A two dimensional memory of one-bit elements
reg cmem [3:0][2:0];
```



VERILOG DATA TYPES

- **Parameter**

Represents a global constant
Declared by a predefined word *parameter*
Example:

```
parameter N = 3;

 //An 4-bit vector
wire [N:0] areg;

//A memory of 4 one-bit elements
reg amem [0:N];

//A memory of four 6-bit
reg [0:5] bmem [N:0];

//A two dimensional memory of one-bit elements
reg cmem [N:0][2:0];
```

LECTURE 3: OVERVIEW

- Verilog data types
- Verilog Operators
- Concurrency
- Typical mistakes in Verilog
- Testbench
- Data flow description
- Concurrent signal assignment + examples
- Assignment with delay + examples
- Concurrent code Verilog
- There will be a 10-minute Quiz assigned on Canvas covering contents up till this lecture. Due Friday at 11pm (No Late submission)

RELATIONAL OPERATORS

The result of these operators are Boolean

- (in)equality (== , !=)
  - It compares 2 values(1,0)
    - The result is unknown (x) if operand are other than 1 and 0
  - Bit by bit comparison
  - If result of condition expression is x
    - Both true and false parts are executed
- (in)equality inclusive (===, !==)
  - It compares all 4 values(1,0,x,z)

Operation	Verilog
Equality	==
Inequality	!=
Less than	<
less than or equal	<=
greater than	>
greater than or equal	>=
equality inclusive	= = =
inequality inclusive	! = =

Example

What is the result of:

- 101x === 1011
- 101x == 1011
- 101x === 101x
- 101z !== 1z00
- 39 == 39
- 14 != 14

ARITHMETIC OPERATORS

C = A operation B

Description	Operator	A and B types	Y type
add, sub, multiplication, division,	+ , - , *, /	A numeric B numeric	numeric
Modulus	mod	A numeric (not real) B numeric (not real)	numeric (not real)
Exponent	**	A numeric B numeric	numeric
concatenation	{ , }	A numeric or array B numeric or array	Same as A
Repetition	{N{A}}	A numeric or array	Same as A

```
reg a = 1'b1; reg b = 2'b00
y = { 4{a}, 2{b} }
Result: y = 8'b11110000
```

SHIFT OPERATORS

C = A shift_operation **1**

	Verilog	C (before)	C (after)
Shift logic left –fills with 0	A << 1	1010	0100
Shift logic right -fills with 0	A >> 1	1010	0101
Shift arithmetic left - fills with LSB	A <<< 1	1001	0011
Shift arithmetic right - fills with MSB	A >>> 1	1010	1101

Red font shows the location of same bit before and after shift
Green font shows the location of new values which is added to the empty location after shifting

LECTURE 3: OVERVIEW

- Verilog datatypes
- Verilog Operators
- Typical mistakes in Verilog
- Testbench
- Data flow description
- Concurrent signal assignment + examples
- Assignment with delay + examples
- Concurrent code Verilog
- There will be a 10-minute Quiz assigned on Canvas covering contents up till this lecture. Due Friday at 11pm (No Late submission)

TYPICAL MISTAKES IN VERILOG

```
module mult_comb (a, b, P)
```

Semicolon (;) is missing at the end of the statement

```
input [1:0] A, b;
```

‘A’ is not defined; it should be lowercase

```
output (3:0) P;
```

Brackets [3:0] should be used instead of parenthesis

```
P[0] = a[0] and b[0];
```

The word ‘assign’ is missing; The word ‘and’ cannot be used in verilog. The logical operator “&” should be used

```
Assign p[0]= S[0] | a[0];
```

S[0] is a vector and not declared; ‘assign’ is a lowercase

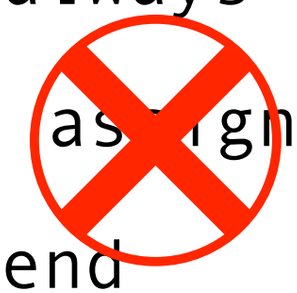
```
endmodule;
```

No semicolon (;)at the end of ‘endmodule’

TYPICAL MISTAKES IN VERILOG

```
reg out;

always @(a,b) begin

 out = a & b;

end
```

```
reg out2;

always @(a,b) begin

 assign out2 = a ^ b;

end
```



There is no ‘assign’ keyword in always blocks!

TYPICAL MISTAKES IN VERILOG

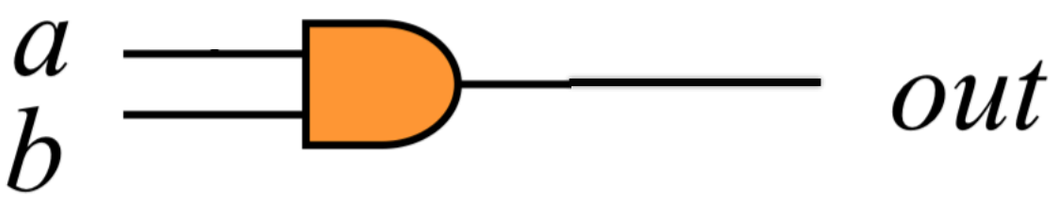
```
module …

.

.

.
  always @(a,b) begin
     x = a^b;
  end
  always @(reset) begin
     x = 1'b0;
  end
endmodule
```

Setting the same `reg` in multiple `always` blocks

- Simulators will typically do what you tell them to do
- Synthesis tools will typically give a warning
- Never do this in Hardware Verilog
- You might be able to get away with it in testing Verilog but normally don't do it

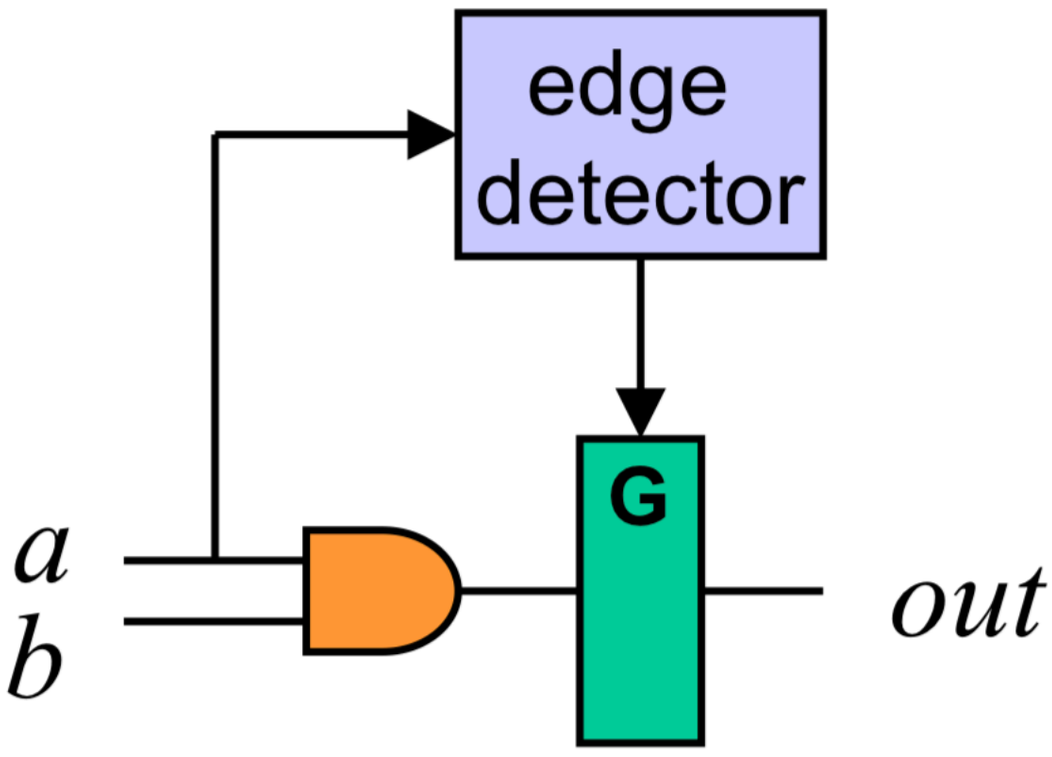
Break 10 minutes

TYPICAL MISTAKES IN VERILOG




```
always @(a) begin // missing b
  out = a & b;
end
```

Result



Incomplete Sensitivity List

- `out` updates when `a` changes as expected
- `out` does not update when be `b` changes!
- Put another way, `b` can change all it wants but `out` will not update - this requires a memory element to remember the last value of `out`
- Using the construct:

  `always @(*)`

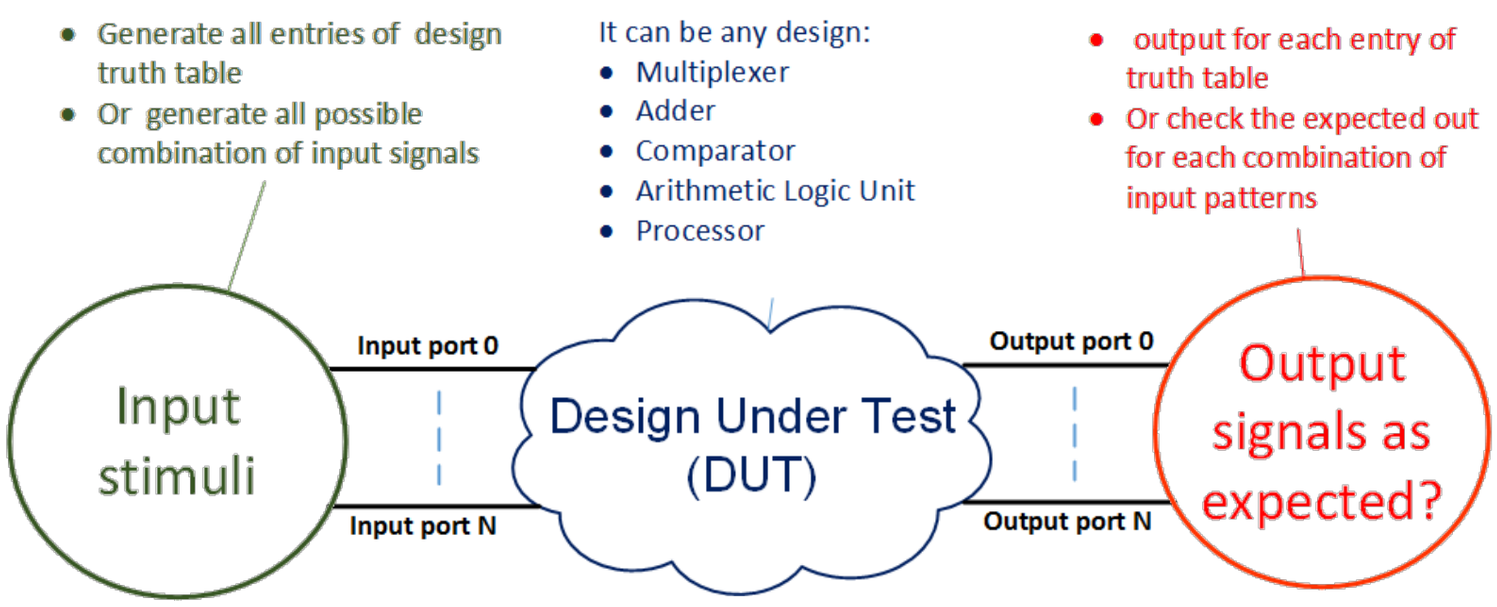
  eliminates this type of bug
- Synthesis tool will give you a warning

LECTURE 3: OVERVIEW

- Verilog datatypes
- Verilog Operators
- Concurrency
- Typical mistakes in Verilog
- Testbench
- Data flow description
- Concurrent signal assignment + examples
- Assignment with delay + examples
- Concurrent code Verilog
- There will be a 10-minute Quiz assigned on Canvas covering contents up till this lecture. Due Friday at 11pm (No Late submission)

Testbench

- Verify a design
  - Instance of Design Under Test
  - Input stimuli
  - Output signals as expected?



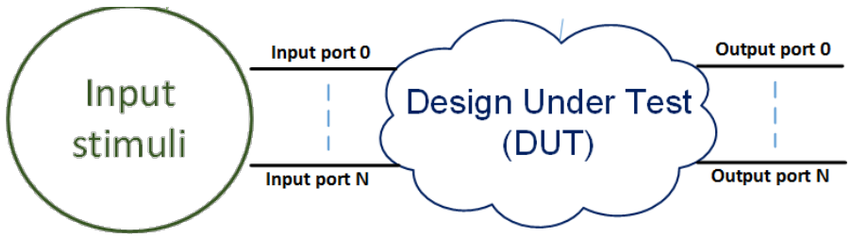
Testbench

- No port definition inside module
- Module is self-contained
  - require no inputs
  - generate no outputs
  - An instance of DUT should be defined
  - For each port of DUT components
    - Signals connected to the DUT inputs (reg) should be defined
    - Signals connected to the DUT outputs (wire) should be defined
- More explanation on difference between reg and wire in the lab session and later today

Testbench for half adder (verilog)

DUT

```
module Half_adder(a,b,S,C);
        input a,b;
        output S, C;
 // Blank lines are allowed

assign S = a ^ b;
//statement 1
assign C= a & b;        //
statement 2
endmodule
```



TB

No port declaration

```
module testbench();
reg in0_tb, in1_tb; // keep the value until they
change - Stimulus
wire s_tb, c_tb; // can read from them - Monitor

Half_adder instant (
.a(in0_tb),
.b(in1_tb),
.S(s_tb),
.C(c_tb));
initial
begin
        in0_tb=1'b0;
        in1_tb=1'b0;
end
always
begin  in0_tb=1'b0;
        in1_tb=1'b0;
        #20
        in0_tb=1'b0;
        in1_tb=1'b1;
        #10
        in0_tb=1'b1;
        in1_tb=1'b0;
        #10
        in0_tb=1'b1;
        in1_tb=1'b1;

end
endmodule
```

Port mapping